

Devoir surveillé terminal

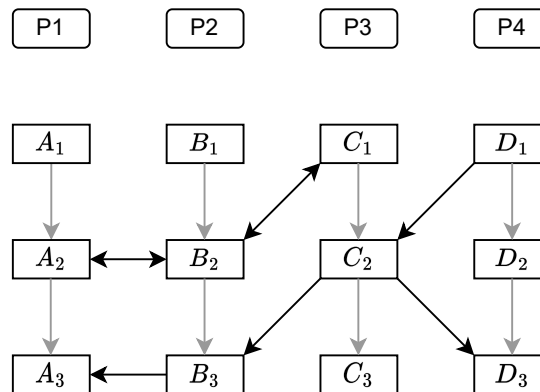
Durée 2h00

Aucun document, outil de calcul ou de communication autorisé. Sauf indications contraires, la gestion d'erreur **ne doit pas** être réalisée et les inclusions ne doivent pas être spécifiées. Dans les portions de code demandées, déclarez les variables utilisées. Toute réponse doit être justifiée.

Exercice 1 (Questions de cours - 6 pts)

1°) Quelle(s) différence(s) et quelle(s) similarité(s) pouvez-vous donner entre une file de messages IPC System V et un tube nommé/anonyme ?

2°) Résolvez le diagramme de précedence suivant en utilisant 2 sémaphores généralisés.



3°) Donnez la portion de code permettant de créer et d'initialiser le tableau de sémaphores pour ce diagramme. La clé est fixée par la constante KEY. Si le tableau existe, un message d'erreur doit être affiché.

Exercice 2 (Autorité de certification - 9 pts)

Nous souhaitons mettre en place le principe de certificats à l'aide de clés privées/publiques. Une application joue le rôle d'autorité de certification (nommée AdC dans la suite). Elle écoute sur deux ports différents : le port X pour la génération de certificats, utilisant des communications via TCP, et le port Y pour la récupération du certificat de l'AdC, utilisant des communications via UDP. Les numéros de port X et Y sont fixés comme constantes dans l'application.

Pour générer un certificat, un client envoie ses informations à l'AdC : un identifiant unique généré à partir du matériel de la machine (de type `uniqueid_t`), un nom (une chaîne de longueur variable) et sa clé publique (de type `public_key_t`). Pour s'assurer que le client est bien le propriétaire de la clé publique, l'AdC envoie un message chiffré avec celle-ci et le client renvoie le message déchiffré. Si c'est correct, l'AdC envoie un certificat signé sa clé privée.

Lorsqu'un client A désire communiquer avec un client B, il envoie son certificat (de type `certificate_t`). Celui-ci peut être vérifié grâce au certificat de l'AdC que le client B peut récupérer à l'aide d'une requête vers l'AdC.

La génération de certificats

1°) Donnez le diagramme de communication entre le client et l'AdC. Précisez les données échangées et leur type, ainsi que la gestion d'erreur.

2°) Est-il nécessaire pour l'AdC de créer un fils pour chaque connexion avec un client ?

3°) Donnez la portion de code de l'AdC permettant de gérer la génération de certificats. Elle doit contenir la création de la socket, la gestion des connexions avec des fils différents et la réception des informations du client. Les autres échanges ne sont pas à faire. La connexion doit être terminée proprement.

4°) Que peut-on dire sur la valeur spécifiée à `listen` ?

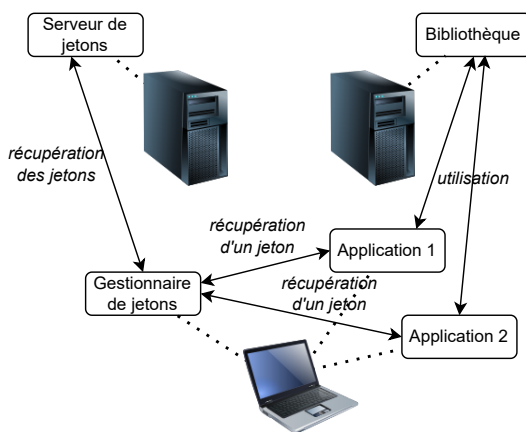
La récupération de certificats

5°) Proposez une solution pour que le serveur puisse récupérer l'adresse du client pour lui répondre.

6°) Que se passe-t-il si le message du client ou la réponse du serveur sont perdus ? Proposez une solution. Précisez les appels systèmes nécessaires.

Exercice 3 (Gestionnaire de jetons - 5 pts)

Nous supposons qu'une bibliothèque distante nécessite un jeton (une chaîne de caractères de 1024 octets) pour être utilisée. Sur une machine qui possède un abonnement, un nombre maximum d'applications peut accéder simultanément à cette bibliothèque.



Pour gérer localement, un gestionnaire récupère la liste des jetons auprès d'un serveur, le nombre dépendant de l'abonnement. Nous ne gérons pas les connexions dans cet exercice.

Le gestionnaire utilise un segment de mémoire partagé pour partager les jetons et leur état (occupé ou libre). Quand une application a besoin d'en utiliser un, elle le marque comme occupé puis se connecte à la bibliothèque (le jeton sert d'authentification). Elle le libère une fois qu'elle a terminée.

1°) Sachant que le nombre de jetons est variable, donnez le contenu du segment de mémoire partagé.

2°) Proposez la structure `segment_t` permettant de mapper le segment.

3°) Donnez le code de la fonction `segment_t *mapping_segment(int key)` où `key` est la clé du segment de mémoire partagé. Elle permet de créer une structure pour mapper le segment.

4°) Quels sont les problèmes de synchronisation rencontrés ici quant à l'accès au segment de mémoire partagé ? Quelle solution proposez-vous ?

Nous souhaitons maintenant gérer le manque de jetons : quand tous les jetons sont occupés, une application demandeuse se met en attente.

5°) Proposez une solution à base de sémaphores en évitant l'attente active.

Annexes : quelques en-têtes de fonctions C

```

int      accept(int sockfd, struct sockaddr *adresseClient, socklen_t *taille);
unsigned int alarm(unsigned int nb_secondes);
int      atexit(void (*procedure)(void));
int      bind(int fd, struct sockaddr *adresse, socklen_t longueur);
int      close(int fd);
int      connect(int sockfd, struct sockaddr *adresseServeur, socklen_t taille);
int      execve(const char *fichier, char *const argv[], char *const envp[]);
void     exit(int statut);
int      fcntl(int fd, int commande, ...);
pid_t    fork();
pid_t    getpid();
pid_t    getppid();
int      getpeername(int sockfd, struct sockaddr *adresse, socklen_t *taille);
int      getsockname(int sockfd, struct sockaddr *adresse, socklen_t *taille);
uint32_t htonl(uint32_t hote_long);
uint16_t htons(uint16_t hote_court);
const char *inet_ntop(int famille, const void *source, char *destination, socklen_t taille);
int      inet_pton(int famille, const char *source, void *destination);
int      kill(pid_t pid, int numero_signal);
int      listen(int sockfd, int taille);
off_t    lseek(int fd, off_t offset, int point_depart);
int      lstat(const char *chemin, struct stat *tampon);
void     memcpy(void *destination, const void *source, size_t taille);
void     memset(void *tampon, int caractere, size_t nombre_elements);
int      mkfifo(const char *nomFichier, mode_t mode);
int      msgctl(key_t cle, int commande, struct msqid_ds *buf);
int      msgget(key_t cle, int options);
int      msgrcv(int msqid, struct msgbuf *msg, size_t taille, long type, int options);
int      msgsnd(int msqid, struct msgbuf *msg, size_t taille, int options);
uint32_t ntohl(uint32_t reseau_long);
uint16_t ntohs(uint16_t reseau_court);
int      open(const char *chemin, int options, mode_t mode);
int      pause(void);
int      pipe(int tube[2]);
ssize_t  recvfrom(int sockfd, void *message, size_t taille, int flags,
                  struct sockaddr *adresse_source, socklen_t *taille_adresse);
ssize_t  read(int fd, void *tampon, size_t taille);
int      semctl(int semid, int semnum, int commande, union semun args);
int      semget(key_t cle, int nbSems, int options);
int      semop(int semid, struct sembuf *sops, unsigned nsops);
ssize_t  sendto(int sockfd, const void *message, size_t taille, int flags,
                const struct sockaddr *adresse_dest, socklen_t taille_adresse);
void     *shmat(int shmid, const void *shmaddr, int options);
int      shmctl(key_t cle, int commande, struct shmid_ds *buf);
int      shmdt(const void *shmaddr);
int      shmget(key_t cle, int taille, int options);
int      sigaction(int num, const struct sigaction *action, struct sigaction *old_action);
int      sigaddset(sigset_t *ensemble, int numero_signal);
int      sigdelset(sigset_t *ensemble, int numero_signal);
int      sigemptyset(sigset_t *ensemble);
int      sigfillset(sigset_t *ensemble);
int      sigismember(sigset_t *ensemble, int numero_signal);
int      sigpending(sigset_t *ensemble);
int      sigprocmask(int comment, const sigset_t *ensemble, sigset_t *ancien);
int      sigqueue(pid_t pid, int numero_signal, const union sigval valeur);
int      sigsuspend(const sigset_t *ensemble);
int      sigwaitinfo(const sigset_t *ensemble, siginfo_t *info);
int      sigtimedwait(const sigset_t *set, siginfo_t *info,
                     const struct timespec *timeout);
unsigned int sleep(unsigned int nombre_secondes);
int      socket(int domaine, int type, int protocole);
int      socketpair(int domaine, int type, int protocole, int sv[2]);
int      unlink(const char *nomFichier);
pid_t    wait(int *statut);
pid_t    waitpid(pid_t pid, int *statut, int options);
ssize_t  write(int fd, const void *tampon, size_t taille);

```

Constantes/macros associées à des paramètres ou des champs de structures

cle (msgget, shmget, semget) : IPC_PRIVATE
commande (fcntl) : F_GETFL, F_SETFL
commande (msgctl, shmctl) : IPC_RMID, IPC_STAT, IPC_SET
commande (semctl) : IPC_RMID, IPC_STAT, IPC_SET, IPC_GETVAL, IPC_GETALL, IPC_SETVAL, IPC_SETALL
comment (sigprocmask) : SIG_BLOCK, SIG_UNBLOCK, SIG_SET
domaine (socket, socketpair) : AF_LOCAL, AF_UNIX, AF_INET, AF_INET6, AF_PACKET
famille (inet_pton, inet_ntop) : AF_INET, AF_INET6
mode (mkfifo) : O_RDONLY, O_WRONLY, O_CREAT, O_EXCL, S_IRWXU, S_IRUSR, S_IWUSR, S_IXUSR...
mode (open) : S_IRWXU, S_IRUSR, S_IWUSR, S_IXUSR...
options (fcntl, open) : O_RDONLY, O_WRONLY, O_RDWR, O_CREAT, O_EXCL, O_TRUNC, O_APPEND
options (msgget, semget, shmget) : IPC_CREAT, IPC_EXCL, S_IRWXU, S_IRUSR, S_IWUSR, S_IXUSR...
options (msgrcv) : IPC_NOWAIT, MSG_EXCEPT
options (msgsnd) : IPC_NOWAIT, MSG_NOERROR
options (shmat) : SHM_RDONLY, SHM_RND
options (waitpid) : WNOHANG, WUNTRACED
point_départ (lseek) : SEEK_SET, SEEK_CUR, SEEK_END
protocole (socket, socketpair) : IPPROTO_TCP, IPPROTO_UDP
sa_flags (champ de struct sigaction) : SA_ONESHOT, SA_NOMASK, SA_SIGINFO
sa_handler (champ de struct sigaction) : SIG_IGN, SIG_DFL
sem_flag (champ de struct sembuf) : IPC_NOWAIT, SEM_UNDO
taille (inet_ntop) : INET_ADDRSTRLEN, INET6_ADDRSTRLEN
type (socket, socketpair) : SOCK_STREAM, SOCK_DGRAM
 Macros sur **statut** de wait, waitpid : WIFEXITED, WEXITSTATUS, WIFSIGNALED, WTERMSIG, WIFSTOPPED, WSTOPSIG

Quelques structures C

```

struct sigaction {
    void (*sa_handler) (int);
    void (*sa_sigaction) (int, siginfo_t*, void*);
    sigset_t sa_mask;
    int sa_flags;
};
union senum {
    int val;
    struct semid_ds *buf;
    unsigned short *array;
};
struct siginfo_t {
    int si_signo;
    int si_code;
    sigval_t si_value;
    pid_t si_pid;
    ...
};
struct msqid_ds {
    struct ipc_perm msg_perm;
    time_t msg_stime;
    time_t msg_rtime;
    time_t msg_ctime;
    msgnum_t msg_qnum;
    msglen_t msg_qbytes;
    pid_t msg_lspid;
    pid_t msg_lrpid;
};
struct sockaddr_in6 {
    sa_family_t sin6_family;
    in_port_t sin6_port;
    uint32_t sin6_flowinfo;
    struct in6_addr sin6_addr;
    uint32_t sin6_scope_id;
};
struct sembuf {
    unsigned short sem_num;
    short sem_op;
    short sem_flg;
};

struct in_addr {
    uint_32 s_addr;
};
struct timespec {
    long tv_sec;
    long tv_nsec;
};
struct semid_ds {
    struct ipc_perm sem_perm;
    time_t sem_otime;
    time_t sem_ctime;
    unsigned short sem_nsems;
};
struct shmid_ds {
    struct ipc_perm msg_perm;
    size_t shm_segsz;
    time_t shm_atime;
    time_t shm_dtime;
    time_t shm_ctime;
    pid_t shm_cpid;
    pid_t shm_lpid;
    shmatt_t shm_nattch;
};
struct sockaddr_in {
    sa_family_t sin_family;
    in_port_t sin_port;
    struct in_addr sin_addr;
};
struct sockaddr_un {
    sa_family_t sun_family;
    char sun_path[108];
};
union sig_val_t {
    int sival_int;
    void *sival_ptr;
};
struct timeval {
    long tv_sec;
    long tv_usec;
};
  
```